Exploratory Data Analysis & Visualisation

A hands-on EDA exercise using a real coffee shop sales dataset.
Load, inspect, analyse, visualise — and tell the story the data is hiding.

DUE DATE Wednesday, 3 June 2026 **PLATFORM** Databricks **SUBMISSION** GitHub link via portal chat

COURSE **Data Analytics with Python**	TYPE **Practical Exercise**
PLATFORM **Databricks (Notebook)**	DATASET **Bright Coffee Shop Sales**

01 About the Dataset

What you are working with

`Bright_Coffee_Shop_Sales.csv` · Rows: **149,116** · Columns: **11** · Period: **January – June 2023**

IMPORTANT — NON-STANDARD CSV FORMAT

This file uses a **semicolon separator** and **comma decimal**. You must specify these when loading:

```
df = pd.read_csv("Bright_Coffee_Shop_Sales.csv", sep=";", decimal=",")
```

Or in Databricks with Spark, then convert: use pandas after `toPandas()`.

Omitting these arguments will raise a parse error. This is **intentional** — real data always has quirks.

Stores: Hell's Kitchen · Astoria · Lower Manhattan

Top categories: Coffee (39%) · Tea (30%) · Bakery (15%) · Drinking Chocolate (8%)

Column	Type	Description
`transaction_id`	*int*	Unique row identifier
`transaction_date`	*str → date*	Date of sale — parse with pd.to_datetime
`transaction_time`	*str → time*	Time of sale (HH:MM:SS)
`transaction_qty`	*int*	Number of items in this transaction line

`store_id`	*int*	Numeric store identifier
`store_location`	*str*	Store name / location
`product_id`	*int*	Numeric product identifier
`unit_price`	*float*	Price per item (decimal comma in source file)
`product_category`	*str*	High-level: Coffee, Tea, Bakery…
`product_type`	*str*	More specific product type
`product_detail`	*str*	Full product name with size

02 Learning Objectives

- Load a real-world dataset with non-default CSV settings in Databricks
- Follow a structured EDA workflow — shape, types, missing values, stats, distributions, correlations, group analysis
- Derive new columns to enable deeper analysis (e.g. revenue)
- Create and interpret visualisations using **matplotlib**, **seaborn**, and **plotly**
- Articulate the trade-offs between the three visualisation libraries
- Extract data-backed business insights from a retail dataset

03 Tasks

Complete all five tasks in a single Databricks notebook

Work through each task in order. Add a **markdown cell** above each section explaining what you are doing and why — this narration is part of the exercise.

Task 01 Load & Inspect the Dataset

1. Load the CSV with the correct `sep` and `decimal` arguments.
2. Print the shape, all column names, and data types.
3. Check for missing values: `df.isnull().sum()`
4. Run `df.describe()` and `df.describe(include='object')`.
5. Print value counts for `product_category` and `store_location`.

Deliverable: *All outputs visible. A markdown cell noting one observation from the summary statistics.*

Task 02 Feature Engineering & Distributions

1. Create a `revenue` column: `unit_price * transaction_qty`.
2. Extract `hour` from `transaction_time` and `month` from `transaction_date`.
3. Plot the distribution of `unit_price` (histogram with KDE). Describe the shape.
4. Plot total revenue by `store_location` as a bar chart.
5. Plot transactions by hour of day. When is the shop busiest?

Deliverable: *Three labelled charts. A markdown cell answering the questions above.*

Task 03 Correlation & Group Analysis

1. Create a seaborn correlation heatmap for `unit_price`, `transaction_qty`, and `revenue`.
2. Build a pivot table: total revenue by `store_location` (rows) × `product_category` (columns).
3. Visualise the pivot table as a grouped or stacked bar chart.
4. Which store earns the most? Which product category drives the most revenue overall?

Deliverable: *Heatmap, pivot table output, and a group bar chart. Written answers to the two questions.*

Task 04 Visualisation Library Comparison

Reproduce the **revenue by product category** bar chart in all three libraries:

- **matplotlib**
- **seaborn**
- **plotly**

Use the reference table in **Appendix A** to guide you.

Written response (4–6 sentences): Which library required the most code? Which had the best default output? In what real-world scenario would you choose plotly over seaborn?

Deliverable: *Three matching charts with identical titles, followed by your written comparison.*

Task 05 Business Insights

Using your EDA findings, write at least **three business insights** that Bright Coffee Shop management could act on. For each insight:

- State the insight clearly in one sentence
- Reference the specific chart or statistic that supports it
- Suggest one concrete action the business could take

> **EXAMPLE — DO NOT REPEAT THIS**
> Coffee accounts for 39% of transactions. If its revenue share is lower, the shop may be under-monetising its most popular category — a premium tier or upsell strategy could help.

Deliverable: *A markdown cell with 3+ clearly formatted insights. Must be specific to this dataset.*

A Appendix — Visualisation Library Reference

Use this when completing Task 04

Feature	Matplotlib	Seaborn	Plotly
Syntax effort	High (verbose)	Low (concise)	Low–Medium
Default look	Basic	Polished	Modern & interactive
Interactivity	None	None	Full (zoom, hover)
Best for	Custom/print figs	EDA & stats	Presentations & apps
pandas integration	Good	Excellent	Good

STARTER CODE SNIPPETS

Load the data:

```
import pandas as pd
df = pd.read_csv("Bright_Coffee_Shop_Sales.csv", sep=";", decimal=",")
df["transaction_date"] = pd.to_datetime(df["transaction_date"])
df["hour"] = pd.to_datetime(df["transaction_time"], format="%H:%M:%S").dt.hour
df["month"] = df["transaction_date"].dt.month
df["revenue"] = df["unit_price"] * df["transaction_qty"]
```

Plotly bar chart (Task 04 starter):

```
import plotly.express as px
cat_rev = df.groupby("product_category")["revenue"].sum().reset_index()
fig = px.bar(cat_rev, x="product_category", y="revenue", title="Revenue by Category")
fig.show()
```

B

Submission

How and when to hand in your work

Due: Wednesday, 3 June 2026, end of day

How to submit:

- Complete your work in a Databricks notebook
- Push the notebook to a GitHub repository
- Post your GitHub repository link in the course portal chat

★ TIPS FOR A STRONG SUBMISSION

- Run all cells top to bottom before submitting — outputs must be visible
- Add a markdown cell above every section explaining what you are doing and why
- Every chart needs a title and labelled axes
- For Task 04, keep the chart type identical across all three libraries for a fair comparison
- Insights must reference specific numbers — "Coffee sells a lot" is not an insight
- `sep` and `decimal` must be set correctly or your load will fail

	Before you submit, confirm:
■	**Notebook** pushed to GitHub and link posted in the portal chat
■	All cells run — outputs visible throughout the notebook
■	Task 01: shape, dtypes, missing values, and describe() outputs shown
■	Task 02: revenue column derived; histogram, bar chart, and hourly chart included

■	Task 03: correlation heatmap, pivot table, and group bar chart included
■	Task 04: same chart in all 3 libraries with written comparison
■	Task 05: at least 3 data-backed business insights with suggested actions
■	Markdown narration added above each section

Good luck! Questions? Reach out before the due date. ■